EDULIBRARY / ТҮПНҰСҚА ОҚУ МАТЕРИАЛЫ

Алгоритмдер және деректер құрылымдары

EduLibrary Demo Content

Алгоритм дұрыстығы, күрделілік, массив, стек, сөздік және граф туралы шағын есептер мен түсіндірмелерден тұратын түпнұсқа оқу материалы.

Қысқаша сабақтар · 2026 · Қазақша

Оқу жолы

Бұл материалды қалай оқу керек

Әр сабақтағы ұғымды оқып, мысалды қолмен жүргізіңіз. Содан кейін тәжірибе тапсырмасын өзіңіз орындап, соңғы беттегі түсіндірмемен салыстырыңыз. Код үзінділері оқу үшін ықшамдалған; олар толық өндірістік жүйені алмастырмайды.

1. Алгоритмді анықтау

2. Уақыт пен жад күрделілігі

3. Массив, стек және кезек

4. Сөздік және бірегейлік

5. Граф және енін бойлай іздеу

Шағын жоба және жауаптарды тексеру

Авторлық құқық және шығу тегі

Автор: EduLibrary Demo Content. 2026 жылы EduLibrary демонстрациясы үшін жаңадан жазылған түпнұсқа қазақша сабақтар. Коммерциялық кітаптан көшірілген мәтін, тарау не файл қолданылған жоқ. Бұл материал бес дербес шағын оқу құралының бірі.

Лицензия: Creative Commons Attribution 4.0 International (CC BY 4.0). Авторды және осы лицензияны көрсетіп пайдалануға болады. Лицензия: https://creativecommons.org/licenses/by/4.0/ . Бастапқы мәтін мен генератор осы жобаның Data/DigitalDemo/content.json және tools/generate_digital_demo.py файлдарында сақталған.

1. Алгоритмді анықтау

Алгоритм - кіріс үшін нәтижеге жеткізетін анық қадамдар тізбегі. Оны жазбас бұрын кіріс, шығыс және шектеулерді белгілеңіз. Ең үлкен мәнді табу есебінде тізім бос болуы мүмкін бе? Барлық мән теріс бола ала ма? Осы сұрақтар дұрыс бастапқы мәнді таңдауға әсер етеді.

Дұрыстықты шағын инвариантпен түсіндіруге болады. Тізімнің алғашқы k элементін қарап болғанда, best сол бөліктегі ең үлкен мән болсын. Келесі элемент үлкен болса best жаңарады, әйтпесе өзгермейді. Осылайша инвариант әр қадамда сақталады. Соңғы қадамда бүкіл тізім қаралып, нәтиже дұрыс болады.

Мысалды талдау

```
best = values[0]
for each value after the first:
    if value > best:
        best = value
return best
```

Өзіңіз орындаңыз

Барлық мән теріс болғанда best=0 деп бастау неліктен қате? [-8,-3,-11] мысалында алгоритмді қолмен жүргізіңіз.

2. Уақыт пен жад күрделілігі

Күрделілік кіріс үлкейгенде жұмыс көлемінің қалай өсетінін сипаттайды. Әр элементті бір рет қарайтын іздеу O(n). Әр элемент үшін барлық қалғанын қарайтын екі ішкі цикл көбіне $O(n^2)$. Бұл секундтың нақты саны емес; құрылғыға тәуелсіз өсу моделін береді.

Тек уақытты емес, қосымша жадты да есептеңіз. Екі тізімді біріктіру үшін жаңа тізім жасау O(n+m) жад алуы мүмкін. Кішкентай кірісте қарапайым алгоритм жылдам әрі түсінікті болуы ықтимал. Өлшеу мен модель бірін-бірі толықтырады: күрделілік үлкен өлшемді болжайды, өлшеу нақты ортадағы шығынды көрсетеді.

Мысалды талдау

```
for i from 0 to n-1:
  print(values[i])      # n operations

for i from 0 to n-1:
  for j from 0 to n-1:
    compare(i, j)     # n*n operations
```

Өзіңіз орындаңыз

n 100-ден 200-ге өссе, сызықтық және квадраттық алгоритмдегі негізгі жұмыс шамамен неше есе артады?

3. Массив, стек және кезек

Массив индекс арқылы элементке тез қатынайды. Ортасына элемент қосу үшін кейінгі элементтерді жылжыту қажет болуы мүмкін. Стек соңғы қосылған элементті бірінші шығарады: LIFO. Кезек бірінші келген элементті бірінші шығарады: FIFO. Құрылымды атауына емес, керек әрекеттерге қарап таңдаңыз.

Мәтін редакторындағы болдырмау тарихы стекке ұқсайды. Кітапты күту өтінімдері кезекке ұқсайды. Бірақ нақты жүйеде жадтағы кезек жеткіліксіз: дерек сақталуы, уақыт бойынша тұрақты рет және қатар өзгерту қорғанысы керек. Алгоритмдік модель домен ережелерін түсіндіреді, іске асыру сол ережелерді толық қорғауы тиіс.

Мысалды талдау

```
queue.enqueue('A')
queue.enqueue('B')
first = queue.dequeue()   # A
stack.push('A')
stack.push('B')
last = stack.pop()        # B
```

Өзіңіз орындаңыз

Жақшалардың дұрыс жабылуын тексеруге неге стек ыңғайлы? '(()())' жолындағы қадамдарды көрсетіңіз.

4. Сөздік және бірегейлік

Сөздік кілтті мәнмен байланыстырады. Мысалы, түгендеу нөмірі бойынша кітап данасын табуға болады. Хеш-кесте негізіндегі сөздік іздеу мен қосуды орташа жағдайда O(1) уақытта орындайды. Бұл абсолют кепілдік емес: коллизиялар, жад және іске асыру ерекшеліктері әсер етеді.

Қайталануды жою үшін жиын ыңғайлы. Кілттің теңдігі анық әрі тұрақты болуы керек. Мәтінді регистрге тәуелсіз салыстырсаңыз, бірізді салыстыру ережесін қолданыңыз. Сөздіктен жоқ кілтті алу қате туғызуы мүмкін, сондықтан қауіпсіз іздеу әдісін пайдаланыңыз. Кілт өзгерсе, бұрынғы байланыс өздігінен мағынасын түземейді.

Мысалды талдау

```
counts = empty dictionary
for word in words:
   key = normalize(word)
   counts[key] = counts.get(key, 0) + 1
return counts
```

Өзіңіз орындаңыз

['SQL','sql','C#','SQL'] үшін регистрге тәуелсіз жиілік кестесін құрыңыз. Қалыпқа келтіруді өткізіп алсақ не өзгереді?

5. Граф және енін бойлай іздеу

Граф түйіндер мен оларды байланыстыратын қырлардан тұрады. Кампус ғимараттарын түйін, жаяу жолдарды қыр ретінде модельдеуге болады. Бағытталған графта A-дан B-ға өту B-дан A-ға өтуді білдірмейді. Салмақсыз графтағы ең аз қыр санымен жететін жолды енін бойлай іздеу, яғни BFS табады.

BFS бастапқы түйінді кезекке салып, көршілерін қабат бойынша қарайды. Қаралған түйіндер жиыны циклге қайта-қайта кіруден қорғайды. Әр түйін үшін алдыңғы түйінді сақтасақ, соңында жолды кері қалпына келтіреміз. Қашықтықтар әртүрлі салмақпен берілсе, қарапайым BFS ең қысқа салмақты жолға кепілдік бермейді.

Мысалды талдау

```
queue = [start]
visited = {start}
while queue is not empty:
   node = queue.dequeue()
   for next in neighbors(node):
      if next not in visited:
         visited.add(next)
         queue.enqueue(next)
```

Өзіңіз орындаңыз

A-B, A-C, B-D, C-D қырлары бар графта A-дан D-ға ең аз қанша қыр қажет? Қаралған түйінді белгілеу неге маңызды?

Жоба және өзіндік тексеру

Кітап атауларының шағын тізімін алыңыз. Сызықтық іздеу, сөздік арқылы іздеу және сөз жиілігін санауды іске асырыңыз. Әр тәсілдің бастапқы дайындау, іздеу және жад шығынын салыстырыңыз. Бос тізім, қайталанатын атау және табылмайтын сұрау жағдайларын тексеріңіз.

Тапсырмаларға түсіндірме

1. 0 тізімде жоқ әрі барлық мәннен үлкен. Бірінші элементтен бастасақ, best -8, кейін -3 болып өзгереді, соңында -3 қалады.

2. Сызықтықта 2 есе, квадраттықта 4 есе. Нақты уақытқа тұрақты шығындар да әсер етеді, бірақ өсу моделі осылай өзгереді.

3. Әр ашық жақша стекке салынып, жабық жақша соңғы ашық жақшаны шығарады. Бос стекке жабық жақша келсе немесе соңында стек бос болмаса, жол дұрыс емес.

4. sql:3 және c#:1. Қалыпқа келтірмесек SQL:2, sql:1, C#:1 болып, мағынасы бір сөз екі кілтке бөлінеді.

5. Екі қыр: A-B-D немесе A-C-D. visited жиыны бір түйінді қайта қосуды шектеп, циклді графта тоқтауды қамтамасыз етеді.

Келесі қадам: жобаңызды басқа адамға түсіндіріп көріңіз. Қандай кіріс қабылданады, қандай ереже қорғалады және нәтижені қалай тексересіз?